

Propuesta de Arquitectura del Proyecto

Plataforma Web de Ayuda Memoria para Cursos de Programación

Tecnologías propuestas

- React
 - TypeScript
 - Chakra UI
 - Visual Studio Code
 - Git y GitHub (para trabajo colaborativo)
-

1. Objetivo del Proyecto

Desarrollar una plataforma web que sirva como **ayuda memoria** para estudiantes de programación.

La plataforma permitirá que, después de cada clase, el estudiante pueda revisar únicamente la información realmente importante, evitando tener que volver a ver una grabación completa para encontrar un concepto específico.

Cada sección de código estará acompañada de una explicación sencilla, ideas clave, recursos para profundizar y un enlace que llevará exactamente al minuto del video donde el profesor explica ese tema.

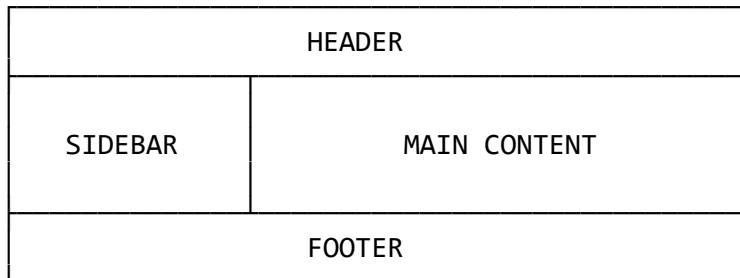
El objetivo principal es facilitar el aprendizaje, priorizando primero la comprensión de la lógica y posteriormente el estudio más profundo de cada tema.

2. Objetivos Específicos

- Organizar el contenido de cada clase de forma clara.
- Mostrar el código por bloques.
- Explicar qué hace cada bloque de código.
- Resumir los conceptos importantes.
- Facilitar el acceso al minuto exacto del video.
- Permitir descargar el código completo de la clase.
- Ofrecer recursos externos para profundizar.
- Permitir que cada estudiante escriba sus propias notas.
- Implementar un buscador por tema o palabra clave.

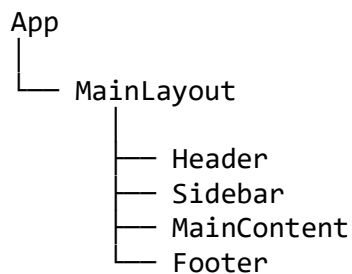
3. Estructura General de la Aplicación

La aplicación estará compuesta por cuatro grandes áreas.



Cada una de estas áreas será desarrollada mediante componentes independientes de React.

4. Arquitectura Basada en Componentes



El componente **MainLayout** únicamente organiza la estructura general de la página.

5. Componentes del Proyecto

5.1 Header

Función

Mostrar la barra superior de la aplicación.

Contenido

- Logo
- Nombre del proyecto

- Buscador
- Usuario (opcional)
- Cambio de tema (modo oscuro/claro, opcional)

Responsabilidad

Organizar la navegación superior y el acceso al buscador.

5.2 Sidebar

Función

Permitir navegar entre todas las clases.

Contenido

- Inicio
- Lista de clases
- Recursos
- Acerca del proyecto

Responsabilidad

Mostrar únicamente la navegación lateral.

5.3 MainContent

Función

Mostrar toda la información correspondiente a la clase seleccionada.

Este componente actuará como contenedor de los demás componentes relacionados con el contenido de la clase.

5.4 Footer

Función

Mostrar información general del proyecto.

Puede contener:

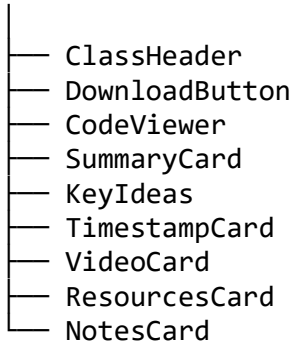
- Versión

- Autores
 - GitHub
 - Año
 - Derechos del proyecto
-

6. Componentes Internos del Contenido

El contenido principal estará dividido en varios componentes pequeños.

MainContent



6.1 ClassHeader

Función

Mostrar la información principal de la clase.

Contenido

- Número de clase
 - Nombre de la clase
 - Nombre del tema
 - Nombre de la sección
-

6.2 DownloadButton

Función

Permitir descargar el código completo desarrollado durante la clase.

6.3 CodeViewer

Función

Mostrar el bloque de código correspondiente.

Características

- Resaltado de sintaxis
- Numeración de líneas
- Botón para copiar código

No tendrá explicaciones; únicamente mostrará el código.

6.4 SummaryCard

Función

Explicar de manera sencilla qué hace el bloque de código y cómo funciona.

El objetivo es que cualquier estudiante pueda comprender la lógica sin leer grandes cantidades de texto.

6.5 KeyIdeas

Función

Mostrar tres ideas fundamentales que el estudiante debe recordar.

Ejemplo:

- No modifica el arreglo original.
 - Ejecuta una función por cada elemento.
 - Devuelve un nuevo arreglo.
-

6.6 TimestampCard

Función

Dirigir al estudiante al minuto exacto del video donde el profesor explica esa parte del código.

Esto evitará buscar manualmente dentro de una grabación extensa.

6.7 VideoCard

Función

Mostrar un reproductor integrado con el video de la clase.

En una versión inicial puede mostrar únicamente la miniatura y un botón para reproducir.

En versiones futuras podrá reproducir el video directamente desde el minuto correspondiente.

6.8 ResourcesCard

Función

Mostrar material complementario.

Ejemplos:

- Documentación oficial
 - MDN
 - JavaScript.info
 - Videos de YouTube
 - Artículos
 - Libros
-

6.9 NotesCard

Función

Permitir que el estudiante escriba la explicación utilizando sus propias palabras.

Esta técnica fortalece el aprendizaje porque obliga a procesar la información y no solo a leerla.

7. Buscador

El buscador permitirá localizar rápidamente información utilizando palabras clave.

Ejemplos:

- map
- useState
- fetch
- props
- arrays

Los resultados mostrarán las clases y secciones donde aparece el concepto buscado.

8. Filosofía del Proyecto

La aplicación no busca reemplazar la clase, sino facilitar el proceso de repaso.

Cada sección debe responder rápidamente cuatro preguntas:

1. ¿Qué hace este código?
2. ¿Cómo funciona?
3. ¿Qué debo recordar?
4. ¿Dónde puedo profundizar?

De esta forma, el estudiante podrá repasar en pocos minutos conceptos que normalmente requerirían revisar largas grabaciones.

9. Propuesta de Distribución del Trabajo (5 Integrantes)

Integrante 1 – Estructura General

Componentes

- MainLayout
- Header
- Footer

Responsabilidades

- Construcción de la estructura principal.
 - Organización de la interfaz.
 - Diseño responsivo del layout.
-

Integrante 2 – Navegación

Componentes

- Sidebar
- SearchBar
- SearchResults

Responsabilidades

- Navegación entre clases.
 - Buscador por temas.
 - Organización del listado de clases.
-

Integrante 3 – Contenido de la Clase

Componentes

- ClassHeader
- DownloadButton
- CodeViewer
- SummaryCard

Responsabilidades

- Presentación del contenido principal.
 - Visualización del código.
 - Resumen explicativo.
-

Integrante 4 – Recursos de Aprendizaje

Componentes

- KeyIdeas
- TimestampCard
- VideoCard
- ResourcesCard

Responsabilidades

- Recursos complementarios.
- Integración con videos.
- Ideas clave y enlaces externos.

Integrante 5 – Notas e Integración

Componentes

- NotesCard
- Integración de componentes
- Gestión de datos compartidos
- Coordinación de pruebas

Responsabilidades

- Integrar el trabajo del equipo.
 - Verificar que todos los componentes funcionen correctamente.
 - Realizar pruebas finales.
 - Resolver conflictos de integración.
-

10. Beneficios de la Arquitectura

Esta arquitectura ofrece las siguientes ventajas:

- Cada integrante trabaja de manera independiente.
 - Se reducen los conflictos al integrar el código.
 - Los componentes pueden reutilizarse en futuras funcionalidades.
 - La aplicación será más fácil de mantener y ampliar.
 - Se sigue la filosofía de React basada en componentes reutilizables.
 - Facilita el trabajo colaborativo mediante Git y GitHub.
-

11. Conclusión

El proyecto se desarrollará siguiendo una arquitectura modular basada en componentes, donde cada elemento tendrá una única responsabilidad. Esta organización permitirá distribuir el trabajo de manera eficiente entre los cinco integrantes del equipo, facilitará la integración del proyecto y hará posible incorporar nuevas funcionalidades en el futuro sin afectar la estructura existente.

El objetivo final es construir una herramienta de estudio clara, intuitiva y escalable, que ayude a los estudiantes a comprender la lógica de programación de forma más rápida y organizada, complementando las clases grabadas con explicaciones, recursos y material de consulta accesible desde una única plataforma.

Archivo Json para generar lo podemos hacer con ia, seria bueno hacerlo con un formulario pero hay mas codigo, primero terminemos la primera parte y si todo funciona y alcanza el tiempo podemos mejorarlo con el formulario para ingresar toda la información

Clase 01

- └─ Título

- └─ Código completo

- └─ Sección 1

- └─ Código

- └─ Resumen

- └─ Ideas clave

- └─ Minuto del video

- └─ Recursos

- └─ Notas (si son personales)

- └─ Sección 2

- └─ Sección 3

Competencia	Criterios	Nivel desempeño	Puntaje Máximo
			20
T	A. Arquitectura y Estructura del Proyecto	4. Cumple con los objetivos	4
	Estructura de carpetas clara y separada por responsabilidad (components, hooks, services/api, types, pages o rutas), siguiendo lo trabajado en clase o una estructura propia igual de sólida y justificable. Tipado correcto con TypeScript en props, estados y respuestas de API (sin abuso de 'any'). Componentización adecuada: evita componentes gigantes, separa lógica de presentación cuando corresponde.		
T, PC	B. Funcionalidades Obligatorias	4. Cumple con los objetivos	5
	El proyecto cumple con los 4 requisitos mínimos obligatorios: 1. Mínimo 3 rutas funcionales con React Router DOM. 2. Login funcional (puede ser simulado o contra una API real). 3. Conexión a 2 APIs (elegidas libremente por el equipo). 4. Manejo de estado global (Context API o Zustand). El puntaje máximo aplica solo si los 4 puntos están completos y funcionando correctamente.		
T, PC	C. Calidad de Código y Buenas Prácticas	4. Cumple con los objetivos	3
	Manejo de estados de carga y error en las peticiones a las APIs. Componentes reutilizables, sin duplicación evidente de lógica. Convenciones de nombres consistentes y código legible. Ausencia de código muerto, comentado sin uso, o console.logs de depuración olvidados.		
TE	D. Trabajo en Equipo y Git Colaborativo	4. Cumple con los objetivos	4
	Uso de ramas por integrante o por feature (no todo el desarrollo en main). Commits distribuidos y descriptivos entre todos los integrantes (no concentrados en una sola persona).		

	Uso de Pull Requests con revisión entre compañeros. Evidencia de resolución de conflictos de merge. Nota: si no hay evidencia real de colaboración en el repositorio, el equipo no puede superar el nivel 'Necesita mejoras', sin importar la calidad del código.		
CE, TE	E. Demostración y Sustentación Oral	4. Cumple con los objetivos	2.5
	Demo en vivo funcional que cubre los flujos principales del sistema (no solo capturas de pantalla). Participación equilibrada de todos los integrantes durante la exposición. Claridad al explicar decisiones técnicas y manejo de preguntas del docente.		
CE	F. Documentación	4. Cumple con los objetivos	1.5
	README claro: instalación, variables de entorno necesarias, cómo correr el proyecto. Breve justificación de decisiones técnicas (por qué esa librería, ese patrón de estado, esa arquitectura).		
G. CONSIDERACIONES COMPLEMENTARIAS (bonus / colchón — no obligatorio, tope de la nota final = 20)			Puntaje Extra
	Deploy del proyecto en Vercel		1
	Uso de una herramienta/librería de diseño no vista en clase (Chakra UI, Radix, shadcn/ui, etc.) (+0.5)		0.5
	Uso de librerías adicionales no enseñadas en clase, bien justificadas (+0.5)		0.5
	Presentación en vivo del proyecto en clase, en lugar de entrega asíncrona (+2)		2